DK_ICSOP_7 - B.Sc. in Computing Systems and Operations

DK_ICCSO_8 – B.Sc. (Hons) in Computing Systems and Operations

Stage 3 - Semester 5

SAMPLE PAPER

COMP I8062 - Algorithms

Instructions

Answer QUESTION ONE (40 marks) and any other TWO questions (30 marks each).

You MUST attempt this question.

Question 1) Algorithm Complexity & Design

(a) Explain the following algorithm design paradigms. For each paradigm, illustrate your answer with a suitable example of an algorithm you have studied.

(i) Backtracking.

(ii) Recursive.

(iii) Two pointer.

(12 marks)

(b) Examine the algorithm found in **appendix A**. It has been designed to process bell-shaped data, i.e. data that ascends to a peak, then decreases.

(i) Assume the algorithm is run with the following array as its input:
data = [8, 15, 16, 21, 23, 30, 29, 28, 16, 14, 12, 8, 6].
What is the result of this algorithm? Explain how this result is reached.

(8 marks)

(ii) What is the paradigm being used for the algorithm's design? Explain the time and space complexity of the algorithm, justifying your answer with reference to specific lines of code.

(6 marks)

(iii) Write and explain the **code** for an alternative (but still functional) solution to this problem that employs recursion instead of iteration. Indicate and explain the base case and recursive case in your implementation.

(14 marks)

Choose any TWO questions from questions 2, 3 and 4

Question 2) Sorting Algorithms

(a) What is meant by stability in sorting? Give an example where stability is required.

(5 marks)

(b) Compare and contrast **Bubble sort** vs **Quick sort** under the following headings:

(i) Time complexity
(ii) Space complexity
(iii) Stability

(15 marks)

(c) Consider the following array: **[22, 18, 5, 8, 10]**
Apply the **Selection Sort** algorithm to sort this array into ascending order. Show the array after each completed pass of the sorting process until it is fully sorted.

(10 marks)

Question 3) Recursive Algorithms

(a) The code shown in **appendix B** is a recursive algorithm to calculate a specified Fibonacci number.

(i) Explain the three essential components required for a recursive function to work correctly. For each component, describe why it is necessary.

(6 marks)

(ii) Identify and explain how each of the three essential components of recursion is implemented in the algorithm shown in **appendix B**.

(6 marks)

(iii) Explain the time and space complexity of the algorithm, justifying your answer with reference to specific lines of code.

(6 marks)

(b) This implementation of the Fibonacci algorithm employs a form of optimisation.

(i) What optimisation technique has been used here? What is the purpose of this form of optimisation? Illustrate your answer with reference to specific lines of code.

(4 marks)

(ii) How does the inclusion of this technique impact the space and time complexities incurred by this algorithm? Explain your answer.

(8 marks)

Question 4) Selecting Algorithms

(a) Your team has been tasked with developing a ticketing system for raising issues internally, with tickets modelled as a queue of Ticket objects stored as an array. The Tickets are to be stored in order of priority, i.e. the Ticket with the highest priority level should always appear first in the queue. You have been assigned to develop a feature that adds a new ticket to the system. The application is envisioned to work on a device with standard power and memory availability, and the application places an emphasis on response time.

(i) Outline two approaches to *array insertion* suitable for use in this context. Ensure that you have adapted them to maintain the sorted nature of the data **without** the need to sort the data after insertion.

(14 marks)

(ii) Which of the two approaches you have discussed would you recommend for use in the scenario above, and why? Justify your answer with evidence based on the manner of operation of the algorithms, and any relevant details from the given scenario.

(6 marks)

(b) Following a caching error, it was discovered that tickets have been added to the ticket queue out of order, rendering the entire ticket queue unsorted. You have been assigned the task of resolving this issue. Rather than sorting the array and continuing as normal, your teammate suggests creating a new array and adding all tickets to that array in sorted order.

(i) Write a short note comparing the performance of your teammate's suggested approach to that of sorting the existing array.

(6 marks)

(ii) Which of the approaches you have discussed would you recommend for use in the scenario above, and why? Justify your answer with evidence based on the manner of operation of the algorithms, and any relevant details from the given scenario.

(4 marks)

APPENDIX A:

```java
public static int altFindPeak(int [] data){
    for (int i = 0; i < data.length-1; i++) {
        if(data[i] > data[i+1]){
            return data[i];
        }
    }

    return data[data.length-1];
}
```

APPENDIX B:

```java
private static final HashMap<Integer, Integer> map = new HashMap<>();
public static int fibonacci(int n){
    System.out.println("Calculating fibonacci #" + n);
    if(n < 0){
        throw new IllegalArgumentException("Fibonacci numbers must be >= 0");
    }

    if(n <= 1){
        return n;
    }

    if(map.containsKey(n)){
        return map.get(n);
    }

    int result = fibonacci( n: n-2) + fibonacci( n: n-1);
    map.put(n, result);

    return result;
}
```